Introduction

In this supplemental commons errors module, we will cover the most common tasking errors we see on this project and how to address them.

Table of Contents

Reminder: Whether you are new to this project or have been tasking for a while, please review the material carefully. Even simple mistakes can create bad training data, and contributors that consistently submit low quality tasks may be removed from the project.

Common Errors

The common errors that we will address in this module are:

1. × Misalignment between golden review and the unified diff.
2. × Rubrics are not self-contained.
3. × Fix-related rubrics are overly restrictive.
4. × Golden Review fails to pass all the rubrics (Response and Rubrics Misalignment)
5. × Prompt, Response and Rubric Misalignment

[Common Error #1: Golden Review and Diff Misalignment]

Why is this considered an error?

The golden review should contain all the serious blocking issues that exist in the unified diff provided in the task artifact. If the unified diff does not support the statements and explanations in the golden review, then the accuracy of the golden review becomes low.

How to avoid this error while tasking

- Make sure that you have reviewed the entire diff provided in the task artifacts.
- Make sure not to add anything based on PR links, commit diff, etc which is not supported by the unified diff in the task artifacts.
- Do not make changes to the diff by yourself to force create issues that do not exist in the unified diff.

× Example of Error

Diff:

```
PLAINTEXT
diff --git a/zerver/data_import/import_util.py b/zerver/data_import/import_util.py
index 481b05ff9c..0f3a4c2080 100644
--- a/zerver/data_import/import_util.py
+++ b/zerver/data_import/import_util.py
.....
..... [These lines contain the code]
.....

diff --git a/zerver/data_import/slack.py b/zerver/data_import/slack.py
index 3bbe2554ec..b757d62f03 100644
--- a/zerver/data_import/slack.py
+++ b/zerver/data_import/slack.py
.....
..... [These lines contain the code]
.....
```

Golden Review (Contains only blocker section):

2. Blockers (must-fix before merge)

- Mutation of input arguments (Side Effect)
  - Whats wrong: In `zerver/data_import/import_util.py`, the `process_uploads` function iterates over `upload_list` and modifies the `path` attribute of each `upload` object (`upload.path = upload_s3_path`).
  - Why it matters: This causes a side effect on the input arguments. The `UploadRecordData` objects passed to the function are permanently altered (the `path` field changes from the source URL to the S3 destination path). If the caller relies on these objects for logging errors, retries, or auditing after this function call, the data will be incorrect. This also violates the principle of separation of concerns during a refactor.
  - Exact Fix: Treat the `upload` object as immutable. Create the dictionary representation first, then update the `path` in the dictionary before appending it to the result list.
  - Where: `zerver/data_import/import_util.py`, lines 53-54 of the provided diff.

- Broken existing tests
  - What's wrong: Two test methods in `zerver/tests/test_slack_importer.py` call `channel_message_to_zerver_message` and then access the returned upload list using dict-key syntax: `uploads[0]["path"]`. After this diff, `channel_message_to_zerver_message` returns `list[UploadRecordData]`, so each item is a dataclass instance, not a dict.
  - Why it matters: Accessing `uploads[0]["path"]` on a dataclass raises `TypeError: 'UploadRecordData' object is not subscriptable`, making both tests fail immediately.
  - Exact fixt: Update the two assertions to use attribute access `uploads[0].path` instead of `uploads[0]["path"]` in both affected test methods.
  - Where: `zerver/tests/test_slack_importer.py`.

Common Error Explanation:

In this task, we can clearly see that the "diff" only contains the code information for two files `import_util.py` and `slack.py` . The golden review contains a sub-heading "Broken existing tests" in the "Blockers" section. In that section, the golden review introduces a new file `test_slack_importer.py` file which is not present in the unified diff. Hence, the golden review is inaccurate and the task can easily fail because of this issue. **Remember**, even if the PR and repo might have that file, but if the **diff** does not contain information on the file, then the golden review should not mention any such issues.

Similarly, if there are claims from `import_util.py` file in the golden review but if those claims are inaccurate based on the codes present in the diff, the task can still fail based on response and rubric misalignment due to inaccurate response claims.

✅ **Corrected Example**

For this task, the corrected version is to completely remove that sub-heading content which includes broken existing tests. If there are cases of inaccurate claims in the golden review, then please fix the golden review to make accurate claims only based on the unified diff.

[Common Error #2: Self-Containment and Over-restrictive issues]

Why is this considered an error?

- If the rubrics are not self-contained, every rubric will be considered as a major rubric error and will contribute towards a task failure.
- If the rubrics are over-restrictive, every rubric will be considered as a moderate rubric error and will contribute towards a task failure.

How to avoid this error while tasking

- Ensure that every rubric is constructed keeping only the golden review in mind to make it self-contained.
- Make sure that the rubrics do not falsely punish valid responses and avoid over-restrictive issues.

× **Example of Error**

Rubric Example:

```
None
The response verifies that clear, actionable feedback is given by specifying
what to change, and why, without hallucinating files that are not in the diff.
```

This criteria mentions “diff” and the files present in the diff. But the model does not have access to those files. Hence, the critera is not self-contained as it does not mention the name of the files and the content it is trying to check in those files.

Rubric Example:

```
None
The response suggests removing the inspector widget from the splitter before
calling `deleteLater()` in the `set_position()` method in
`qutebrowser/browser/inspector.py`.
```

The criteria mentions a valid solution to fix the issue present in the **`set_position()`** method. But it clearly punishes other valid possible solutions to resolve the similar issue. Hence, the criteria become overly restrictive.

✅ **Corrected Example**

Fixed rubric (self-containment issue)

```
None
The response verifies that the ValueError handling test case is added on the
file import_logic_values.py.
```

In this corrected version, the criteria clearly mentions the test case, issue being checked and the exact file names which are all mentioned in the golden review response as well. Hence, the criteria is based on the golden response and the explanation included in the golden review response.

Fixed rubric (overly restrictive issue)

```
None
The response suggests a fix for Blocker: the Inspector widget is not removed
from splitter when closed, for example, removing the inspector widget from the
splitter before calling `deleteLater()` in the `set_position()` method in
`qutebrowser/browser/inspector.py`.
```

In this corrected version, the criteria follows a format to mention the possible solution/fix as one of the examples (one of the many possible solutions) and hence the criteria is not overly restrictive.

[Common Error #3: Prompt, Response and Rubric Misalignment]

Why is this considered an error?

The response should fulfill the prompt requirements clearly. If the response has inconsistencies with the requests made in the prompt or if the prompt is not aligned based on the unified diff, there are chances of having a bad golden review response. Similarly, the response must pass all the rubrics and if there are inconsistencies in rubrics with respect to the golden response, the task can be a fail.

How to avoid this error while tasking

1. Make sure the prompt is written solely based on the unified diff.
2. Ensure that the response fulfills the prompt and is aligned with the unified diff.
3. Ensure that the response passes all the rubrics criteria created in the task.

× Example of Error

Prompt:

You are a senior software engineer reviewing a pull request diff for the financial-asset-relationship-db repository, a comprehensive 3D visualization systam for interconnected financial assets. Your role is to verify that the changes correctly solve the stated problem, are production-ready, follow Python and pytest best practices, and do not introduce regressions or silently reduce test coverage.

This PR is titled "refactor: remove unused nested functions and classes." The test file `` `test/unit/test_database.py` `` currently defines several helped functions nested inside methods as well as inline engine creation scattered across test classes. The goal of this refactor is to eliminate these unused or unnecessary nested construct, extract shared setup into proper pytest fixtures, and modernize the test style (removing `` `@staticmethod` `` decorators, adding type annotations) without reducing the effective coverage of the test suite. The refactored tests must continue to verify engine creation, session factory behavior, database initialization, transactional session scope (commit rollback, error propagation, session closure), default URL configuration, and edge cases.

A full unified diff is attached as a file named diff.txt. Review diff.txt as the sole source of code changes. Do not search for or locate the PR externally. Assume all necessary context is provided in this task and the attached diff.

Return a detailed PR review with issues, risks, and required changes before merge. Provide specific, actionable feedback and cite file pats ands line references where possible based on the diff.

Output format (required):

1. Summary
2. Blockers (must-fix): for each, state what's wrong, why it matters, and how to fix it
3. Non-blocking suggestions
4. Tests/validation: include verification that no test coverage was silently dropped, that renamed or restructured tests still assert the same behaviors as before, and that fixture isolation does not introduce new failure modes.

Rubric Example:

16 **The model should flag that the refactor's scope exceeds the issue title "refactor: remove unused nested functions and classes".**

Important

In this task example:

- The prompt does not ask to flag anything outside the refactor scope (which is fine because the core request of code review is present).
- The response also does not explicitly flag anything or have any statement about flagging anything outside the refactor scope.
- But the rubric criteria about flagging outside the refactor scope is written and is also marked as important.

  Hence, rubric criteria written is inconsistent with the response as well as the prompt in this example. Also, the golden response does not pass this rubrics criteria at all making the task as a Fail.

✅ **Corrected Example**

- The correction here should be to either remove this rubric criteria (making sure the rubrics count is 15 or more).
- Add the explicit flag as part of the response and mark the rubric criteria as Nice to Have because it is not a core requirement.